Besides counting objects in memory, .size() also uses a Counter Cache column when it's available which avoids issuing a SQL COUNT() query.

Counter Cache columns include various trade-offs. They require a new column be added and they also add a small bit of latency at write time. Counter caches do open up the possibility of inconsistent counts. A high volume of UPDATE statements to the same counter cache column can also cause database bloat which you'll learn more about later.

Despite these downsides, counter caches are a great performance tactic in many situations because they provide nearly instant counts without SQL COUNT() queries. Read more in *Caching counters with ActiveRecord's counter caches*.[29]

The following Migration for Rideshare added a trips_count column to the users table to act as a counter cache.

sh/add_counter_cache.sh

```
# create the migration
bin/rails g migration AddTripsCountToUsers trips_count:integer

# run the migration
bin/rails db:migrate
```

The counter_cache: true option was added to the :driver association in the Trip model:

```
class Trip < ApplicationRecord
  belongs_to :driver, counter_cache: true
end
```

When adding a new counter cache to a production system, the counter cache will not be populated for existing rows. You'll need to populate the cached value before you can use it or it will return 0.

To explore how the Counter Cache works for a driver, run the following code from bin/rails console.

ruby/reset_counters.rb

```
driver = Trip.first.driver

driver.trips.count

Driver.reset_counters(driver.id, :trips)

driver.trips.size
```

This code gets a Driver, and queries for the Trips count. Notice it makes a SQL query. Next it resets the Trips Count Counter Cache for the Driver. By

29. https://blog.appsignal.com/2018/06/19/activerecords-counter-cache.html